

# Control de Lectura

Capítulo 11 de Eloquent Javascript

Ing. Rodolfo Christian Catunta Uturunco

¿Qué significa que JavaScript sea un lenguaje de ejecución  
síncrona pero con capacidades asíncronas?

¿Por qué las operaciones lentas, como leer archivos o hacer peticiones de red, serían problemáticas si JavaScript solo trabajara de forma completamente síncrona?



# Explica el siguiente gráfico

synchronous, single thread of control



synchronous, two threads of control



asynchronous



¿Qué problema intenta resolver la programación asíncrona dentro de aplicaciones modernas y servidores backend?

¿Qué diferencia conceptual existe entre “esperar bloqueando el programa” y “registrar una acción para ejecutarse después”?



¿Por qué los callbacks/retrollamadas fueron una solución útil para la asincronía, pero también pueden volver el código difícil de leer?

¿Qué ventaja ofrecen las Promises frente al uso tradicional de callbacks/retrollamadas?



¿Cuál es la diferencia entre usar `.then()` y `.catch()` en una Promise?

¿Por qué lanzar errores dentro de una Promise no funciona exactamente igual que en código síncrono?

¿Qué problema resuelve `async/await` y por qué muchas personas consideran que hace el código más legible?



¿Qué ocurre realmente detrás de escena cuando usamos  
`await` dentro de una función `async`?

¿Qué riesgos o errores pueden aparecer cuando múltiples tareas asíncronas interactúan con los mismos datos o recursos al mismo tiempo?

¿Cómo ayuda `Promise.all()` a enviar varias solicitudes  
asíncronas al mismo tiempo y esperar a que todas terminen  
antes de continuar?



¿Por qué el capítulo afirma que el comportamiento asíncrono ocurre con una pila de llamadas vacía, y cómo se relaciona esto con la dificultad de capturar errores usando `try/catch` tradicional?